PYTHON PROGRAMMING

COMPLETE HANDBOOK — BASIC TO ADVANCED

PART 10 — FINAL SECTION

Interview Questions · MCQs · Practice Questions

Cheat Sheet · Roadmap 2026 · Master Revision Notes

Completes the Handbook (Parts 1-9, Modules 1-25)

TABLE OF CONTENTS

ABOUT THIS FINAL SECTION

This closing section consolidates the whole handbook into exam- and interview-ready reference material: a large bank of interview questions and MCQs organised by topic, coding practice problems by difficulty, a one-stop syntax cheat sheet, a suggested 2026 learning roadmap, and master revision notes spanning every module. Use it in the final weeks before an interview or exam as a focused review.

SECTION A: PYTHON INTERVIEW QUESTIONS

A curated bank of interview questions spanning every module of this handbook, grouped by topic for focused review. (Topic-specific Q&A sets already appear at the end of each module — this section adds cross-cutting and frequently-asked questions not repeated elsewhere.)

A.1 Python Basics & Syntax

Q1. What makes Python 'dynamically typed'?

Ans: Variable types are determined automatically at runtime based on the assigned value, and a variable can be reassigned to a different type later — there is no compile-time type declaration.

Q2. What is PEP 8?

Ans: Python Enhancement Proposal 8 — the official style guide for writing readable Python code, covering naming conventions, indentation, line length, and more.

Q3. What is the difference between == and is, once more, in a mixed-type context?

Ans: == checks value equality (works across compatible types, e.g., 1 == 1.0 is True). 'is' checks whether two names refer to the exact same object in memory, regardless of value.

Q4. What are mutable and immutable objects? Give two examples of each.

Ans: Mutable objects can be changed after creation (list, dict, set). Immutable objects cannot (int, str, tuple, frozenset) — any 'change' actually creates a new object.

Q5. What is the difference between a shallow copy and a deep copy?

Ans: A shallow copy (copy.copy() or list.copy()) copies the outer object but still references the same nested objects inside. A deep copy (copy.deepcopy()) recursively copies every nested object too, so the two are fully independent.

Q6. What is duck typing?

Ans: A Python philosophy where an object's suitability is determined by the methods/behaviour it has, not its explicit type — 'if it walks like a duck and quacks like a duck, treat it as a duck'.

Q7. What is the Global Interpreter Lock (GIL)?

Ans: A mutex in CPython that allows only one thread to execute Python bytecode at a time, even on multi-core systems — it simplifies memory management but limits true parallel CPU-bound multithreading (multiprocessing is used instead for that).

Q8. What is the difference between a compiled and interpreted language, and where does Python fit?

Ans: Compiled languages convert source code entirely into machine code before running (e.g., C++). Interpreted languages execute code line-by-line via an interpreter. Python compiles to bytecode internally, then interprets that bytecode via the PVM — a hybrid approach.

A.2 Data Structures

Q1. Why might you choose a set over a list for membership testing?

Ans: Checking 'x in collection' is on average O(1) for a set (hash-based) versus O(n) for a list (linear scan) — sets are far faster for large collections when you only need to check membership.

Q2. What is the time complexity of appending to a Python list?

Ans: Amortised O(1) — Python lists over-allocate memory, so most appends are instant; occasionally the list must be resized/copied, but this happens rarely enough that the average stays constant time.

Q3. How would you remove duplicates from a list while preserving order?

Ans: Using dict.fromkeys(lst) (Python 3.7+, since dicts preserve insertion order) then converting back to a list — this removes duplicates without the ordering loss that a plain set() conversion would cause.

Q4. What is a frozenset?

Ans: An immutable version of a set — once created, it cannot be modified, which also makes it hashable and usable as a dictionary key or set element, unlike a regular set.

Q5. When would you choose a dictionary over a list of tuples?

Ans: When you need fast lookups by a unique key (O(1) average) rather than searching linearly through a list — e.g., looking up a student's record by ID.

A.3 Functions & OOP

Q1. What is a closure?

Ans: A function that remembers and has access to variables from its enclosing (outer) function's scope, even after the outer function has finished executing — created when an inner function is returned/passed out of an outer function.

Q2. What is the difference between a class method, static method, and instance method?

Ans: An instance method takes self and operates on a specific object. A class method (@classmethod) takes cls and operates on the class itself, shared across all instances. A static method (@staticmethod) takes neither — it's a regular function logically grouped inside the class.

Q3. What is multiple inheritance, and what is the MRO?

Ans: Multiple inheritance is when a class inherits from more than one parent class. The MRO (Method Resolution Order) is the specific order Python searches through parent classes to find a method — determined by the C3 linearisation algorithm, viewable via ClassName.__mro__.

Q4. What is the difference between __str__ and __repr__?

Ans: __str__ defines a readable, user-friendly string representation of an object (used by print()). __repr__ defines an unambiguous, developer-facing representation (ideally one that could recreate the object) — used in the interactive shell and by default if __str__ isn't defined.

Q5. Why is 'self' explicitly required as the first parameter in Python methods, unlike some other languages?

Ans: Python's design philosophy favours explicitness — self makes clear that instance methods operate on a specific object, and it is automatically passed by Python whenever you call obj.method(), even though you don't include it in the call itself.

A.4 Advanced Python & Libraries

Q1. What is the difference between deep learning libraries and NumPy/Pandas in the Python ecosystem?

Ans: NumPy/Pandas focus on general numerical computing and data manipulation. Deep learning libraries (TensorFlow, PyTorch) build on similar array concepts but add automatic differentiation and GPU acceleration specifically for training neural networks.

Q2. What is a context manager, and how does 'with' use it?

Ans: An object implementing __enter__ and __exit__ methods, used to reliably set up and tear down resources (like opening/closing a file). The 'with' statement calls __enter__ at the start and guarantees __exit__ runs at the end, even if an exception occurs.

Q3. What is the difference between multiprocessing and multithreading in Python?

Ans: Multithreading runs multiple threads within one process, but the GIL limits true parallel CPU-bound work. Multiprocessing runs separate processes, each with its own Python interpreter and memory (bypassing the GIL), making it better suited for CPU-intensive parallel tasks.

Q4. What is pip, and what problem does a virtual environment solve?

Ans: pip is Python's package installer, used to install libraries from PyPI. A virtual environment (venv) creates an isolated Python installation per project, preventing dependency version conflicts between different projects on the same machine.

Q5. What is the difference between a Python script and a Python module?

Ans: In practice, they're the same kind of file (.py) — the distinction is about usage: a 'script' is run directly to perform a task, while a 'module' is written to be imported and reused by other files.

SECTION B: MULTIPLE CHOICE QUESTIONS

Additional cross-topic MCQs for exam practice, building on the module-specific MCQ sets found throughout the handbook.

B.1 Basics & Data Types

No.	Question	Answer	Options (A-D)
1	Which of these is an immutable type?	**C**	A) list B) dict C) tuple D) set
2	type(True) returns:	**A**	A) <class 'bool'> B) <class 'int'> C) <class 'str'> D) Error
3	Which is the correct way to check a variable's type?	**B**	A) typeof(x) B) type(x) C) x.type() D) gettype(x)
4	0.1 + 0.2 == 0.3 in Python evaluates to:	**B**	A) True B) False C) Error D) None
5	Which operator returns the remainder of division?	**C**	A) / B) // C) % D) **

B.2 Data Structures

No.	Question	Answer	Options (A-D)
1	Which is the fastest for membership testing (x in y)?	**D**	A) list B) tuple C) string D) set
2	dict.fromkeys([1,2,2,3]) has how many keys?	**B**	A) 4 B) 3 C) 2 D) 1
3	Which method adds an item at a specific list index?	**A**	A) insert() B) append() C) add() D) put()
4	A frozenset is:	**C**	A) A mutable set B) A sorted set C) An immutable set D) A duplicate-set
5	Which data structure maintains insertion order by guarantee since 3.7?	**B**	A) set B) dict C) frozenset D) None of these

B.3 Functions & OOP

No.	Question	Answer	Options (A-D)
1	Which decorator defines a method that doesn't need self or cls?	**D**	A) @classmethod B) @property C) @abstractmethod D) @staticmethod
2	A closure requires:	**A**	A) A nested function referencing an outer variable B) Two classes C) A decorator D) A global variable only
3	__str__ is primarily used by:	**C**	A) type() B) isinstance() C) print() D) id()

No.	Question	Answer	Options (A-D)
4	Multiple inheritance means a class:	**B**	A) Has multiple objects B) Inherits from more than one class C) Has multiple constructors D) Cannot be instantiated
5	Which keyword defines a context manager's cleanup step?	**C**	A) __enter__ B) __del__ C) __exit__ D) __close__

B.4 Advanced & Libraries

No.	Question	Answer	Options (A-D)
1	Which module handles isolated project dependencies?	**A**	A) venv B) pip only C) sys D) os
2	GIL stands for:	**B**	A) Global Import Lock B) Global Interpreter Lock C) General Interface Layer D) Global Index List
3	Which is better suited for CPU-bound parallel tasks in Python?	**D**	A) Threading B) Async/await C) Single loop D) Multiprocessing
4	Which function loads a JSON string into a Python object?	**C**	A) json.dump() B) json.dumps() C) json.loads() D) json.read()
5	Which library is most associated with tabular data analysis?	**B**	A) NumPy B) Pandas C) Matplotlib D) requests

SECTION C: CODING PRACTICE QUESTIONS

50 practice problems spanning the whole handbook, grouped by difficulty. Solve each without looking at earlier module solutions first, then check your logic against the relevant module.

Easy (1-20)

1. Check if a number is positive, negative, or zero.
2. Print all multiples of 3 between 1 and 50.
3. Swap the values of two variables.
4. Find the sum of digits of a number.
5. Check if a string is empty.
6. Convert Celsius to Fahrenheit.
7. Find the length of a list without using len().
8. Print the first and last element of a list.
9. Check if a number is divisible by both 5 and 11.
10. Count uppercase and lowercase letters in a string.
11. Find the area of a triangle given its base and height.
12. Print the ASCII value of a character.
13. Check whether a list is empty.
14. Concatenate two strings without using +.
15. Find the average of numbers in a list.
16. Print numbers from 10 to 1 in reverse using a loop.
17. Check if a character is a vowel.
18. Convert a list of strings to a list of integers.
19. Find the smallest of three numbers.
20. Print a multiplication table for a given number.

Medium (21-35)

21. Write a program to check if a number is an Armstrong number.
22. Find all prime numbers between 1 and 100 using a function.
23. Write a program to flatten a nested list (one level deep).
24. Count the frequency of each element in a list without using Counter.
25. Write a program to check if two strings are rotations of each other.
26. Implement a simple stack using a list (push, pop, peek).
27. Write a program to find the second most frequent word in a paragraph.
28. Merge two sorted lists into one sorted list without using sort().
29. Write a function to check if a number is a perfect number.
30. Implement binary search on a sorted list.
31. Write a program to find all pairs in a list that sum to a target value.
32. Write a program to convert a Roman numeral to an integer.

33. Implement a simple queue using a list (enqueue, dequeue).
34. Write a program to remove all duplicate characters from a string, keeping first occurrence.
35. Write a function that returns the transpose of a matrix (nested list).

Hard (36-50)

36. Implement a basic LRU (Least Recently Used) cache using OrderedDict.
37. Write a recursive function to solve the Tower of Hanoi and print each move.
38. Implement quicksort on a list without using the built-in sort().
39. Write a program to detect if a linked list (simulated with a list of nodes) has a cycle.
40. Build a simple text-based calculator that correctly respects operator precedence using a stack.
41. Write a program to find the longest common subsequence of two strings.
42. Implement a basic graph using an adjacency list and perform BFS traversal.
43. Write a decorator that caches (memoizes) the results of a slow recursive function.
44. Write a generator that yields prime numbers indefinitely (an infinite sequence).
45. Implement a simple JSON validator that checks if brackets/braces are balanced in a string.
46. Write a program to serialize and deserialize a simple binary tree using JSON.
47. Build a basic command-line inventory system with persistent SQLite storage.
48. Implement the Sieve of Eratosthenes to find all primes below N efficiently.
49. Write a multi-threaded program to download multiple URLs concurrently (using threading).
50. Build a simple recommendation function that suggests items based on shared tags (set intersection).

SECTION D: PYTHON CHEAT SHEET

D.1 Data Types & Conversion

```
int(x), float(x), str(x), bool(x), list(x), tuple(x), set(x), dict(x)
type(x)          -> returns the type of x
isinstance(x, T)  -> checks if x is of type T
```

D.2 String Cheat Sheet

Syntax	Meaning	Example	Result
s[i]	Character at index i	'hi'[0]	'h'
s[a:b]	Slice from a to b-1	'hello'[1:3]	'el'
s.upper()/.lower()	Case conversion	'Hi'.upper()	'HI'
s.strip()	Remove outer whitespace	' hi '.strip()	'hi'
s.split(sep)	Split into a list	'a,b'.split(',')	['a','b']
sep.join(list)	Join list into string	','.join(['a','b'])	'a,b'
s.replace(a,b)	Replace substring	'cat'.replace('c','b')	'bat'
f'{var}'	f-string formatting	f'{5+5}'	'10'

D.3 List / Tuple / Set / Dict Cheat Sheet

Syntax	Meaning	Example	Result
lst.append(x)	Add to end	[1,2].append(3)	[1,2,3]
lst.pop()	Remove & return last	[1,2,3].pop()	3
[x for x in y if c]	List comprehension	[x*2 for x in [1,2]]	[2,4]
t = (1,2)	Tuple (immutable)	t[0]	1
s = {1,2}; s.add(3)	Set add	{1,2}.add(3)	{1,2,3}
d.get(k, default)	Safe dict access	{}.get('x',0)	0
d.items()	Key-value pairs	{'a':1}.items()	[('a',1)]

D.4 Control Flow & Functions

```
if cond:            elif cond2:            else:
for x in seq:       while cond:            break / continue / pass

def f(a, b=1, *args, **kwargs):
    return a + b

lambda x: x * 2

try:
    ...
except SomeError as e:
    ...
else:
    ...
finally:
    ...
```

D.5 OOP Quick Reference

```
class MyClass(ParentClass):
    def __init__(self, x):
        self.x = x            # instance attribute

    def method(self):          # instance method
        return self.x

    @staticmethod
    def static_method():        # no self/cls needed
        pass

    @classmethod
    def class_method(cls):      # cls refers to the class itself
        pass
```

D.6 File Handling & Common Imports

```
with open("file.txt", "r") as f:
    data = f.read()

import math, random, os, sys, json, csv, re, datetime
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
```

SECTION E: PYTHON ROADMAP 2026

A suggested path for progressing from complete beginner to job-ready, mapped to the modules in this handbook.

Stage	Timeframe (approx.)	Focus & Modules
1. Foundations	Weeks 1-3	Modules 1-6: syntax, data types, operators, control flow, loops, strings. Goal: comfortably write small scripts.
2. Data Structures	Weeks 4-5	Modules 7-10: lists, tuples, sets, dictionaries. Goal: choose the right structure for a problem confidently.
3. Core Programming	Weeks 6-8	Modules 11-13: functions, modules/packages, exception handling. Goal: write modular, reusable, robust code.
4. OOP & Files	Weeks 9-11	Modules 14-16: file handling, OOP, advanced Python (generators, decorators, regex). Goal: build a beginner project (Module 25).
5. Data Track	Weeks 12-15	Modules 17-19: NumPy, Pandas, Matplotlib. Goal: analyse and visualise a real dataset end-to-end.
6. Connectivity	Weeks 16-18	Modules 20-22: databases, web scraping, APIs. Goal: build an app that reads/writes real external data.
7. Automation & ML Intro	Weeks 19-21	Modules 23-24: automation, ML basics. Goal: automate a real repetitive task; train a first ML model.
8. Portfolio & Interviews	Weeks 22-24	Module 25 (Intermediate/Advanced projects) + this Final Section. Goal: 3-5 portfolio projects, mock interviews using the Q&A banks.

How to Use This Roadmap

Timeframes assume roughly 1-2 hours of focused daily practice; adjust based on your own pace. The most important habit is consistency: type out and run every example yourself rather than only reading — that's what actually builds fluency.

SECTION F: MASTER COMMON ERRORS REFERENCE

A consolidated quick-reference of the most frequent errors beginners hit, spanning the whole handbook (each module's own section covers these in more depth with context).

Error	Typical Cause	Fix
SyntaxError	Missing colon, unmatched bracket/quote	Re-read the exact line the traceback points to
IndentationError	Inconsistent spaces/tabs	Use 4 spaces consistently, never mix with tabs
NameError	Variable used before assignment, or typo	Check spelling and definition order
TypeError	Operation on incompatible types	Convert types explicitly before operating
ValueError	Right type, invalid value (e.g., int('abc'))	Validate input before converting
IndexError	Index beyond sequence length	Check len() before indexing
KeyError	Missing dictionary key	Use .get() or check 'in' first
AttributeError	Calling a method that doesn't exist on that type	Check the object's actual type with type()
ZeroDivisionError	Dividing by zero	Check divisor before dividing
ModuleNotFoundError	Library not installed / misspelled	pip install the package; check spelling
RecursionError	Missing/unreachable base case	Verify the base case is correct and reachable
FileNotFoundError	Wrong path or file doesn't exist	Check the path; use 'w'/'a' mode to create if needed

SECTION G: MASTER QUICK REVISION NOTES

A single condensed pass over the entire handbook — use this as your final review before an exam or interview.

Basics & Control Flow

- Python is interpreted, dynamically typed, and object-oriented. Indentation defines blocks.
- Core types: int, float, str, bool, list, tuple, set, dict, NoneType.
- if/elif/else, for, while, break, continue, pass, match-case are the core control-flow tools.

Data Structures

- list [mutable, ordered], tuple (immutable, ordered), set {unique, unordered}, dict {key:value, insertion-ordered since 3.7}.
- List comprehensions: [expr for x in seq if cond].

Functions & OOP

- def, return, *args/**kwargs, lambda, recursion (needs a base case), local vs global scope.
- class, __init__, self, encapsulation (__var), inheritance, polymorphism, abstraction (abc module).

Files, Errors & Advanced Features

- Always use 'with open(...) as f:'. Modes: 'r' read, 'w' write (erases!), 'a' append.
- try/except/else/finally for error handling; raise for custom exceptions.
- Generators (yield) are memory-efficient; decorators (@decorator) wrap functions; re module for pattern matching.

Libraries & Real-World Skills

- NumPy: fast arrays, vectorised math. Pandas: Series/DataFrame, groupby, cleaning. Matplotlib: line/bar/pie/histogram/scatter.
- sqlite3/MySQL for databases (always parameterise queries); requests + BeautifulSoup for scraping; requests + .json() for APIs.
- Automation with os/shutil/schedule/smtplib; ML basics with scikit-learn (train_test_split, fit, predict).

CLOSING NOTE

That completes the Python Programming Complete Handbook — from your very first print("Hello, World!") through data structures, functions, OOP, file handling, the NumPy/Pandas/Matplotlib data stack, databases, web scraping, APIs, automation, machine learning basics, and eleven complete hands-on projects.

The single most important habit for turning this material into real skill is consistent, active practice: type out every example, deliberately break things to see the errors, and build your own variations of each project. Revisit the interview questions and MCQs in this final section periodically — spaced repetition is far more effective than a single read-through.

Good luck with your studies, your interviews, and everything you build with Python next.